

Análise de Desempenho de Servidores RTMP e Transcodificadores no Streaming de Vídeo

Versão inicial em LaTeX para o TCC

Nome do(a) autor(a)

18 de maio de 2026

Resumo

Este artigo apresenta a base inicial de um Trabalho de Conclusão de Curso sobre a comparação de desempenho entre servidores RTMP e transcodificadores em uma plataforma de streaming ao vivo. O projeto utiliza uma arquitetura baseada em Docker, backend em Spring Boot, frontend em React e uma matriz experimental com Nginx-RTMP e SRS, combinados com FFmpeg e GStreamer. Os resultados preliminares do benchmark mais recente já estão organizados em formato reutilizável para redação, análise comparativa e rastreabilidade metodológica.

Conteúdo

Resumo	2
1 Introdução	2
1.1 Objetivo geral	2
1.2 Objetivos específicos	2
2 Fundamentação técnica	3
2.1 Leitura correta das métricas	3
3 Metodologia experimental	3
3.1 Recalibração dos critérios de aceitação	4
3.2 Matriz de comparação	4
3.3 Fontes de dados	5
4 Ambiente de testes e resultados preliminares	5
4.1 Como ler os resultados de falha	5
5 Discussão	6
6 Conclusão	7

Resumo

O objetivo deste trabalho é analisar o desempenho de combinações de servidores RTMP e transcodificadores em cenários de streaming de vídeo ao vivo, com foco na latência de disponibilização do HLS, no uso de CPU e memória do container de ingestão, na taxa de erros e na estabilidade do pipeline. O ambiente foi estruturado com Docker Compose, backend em Spring Boot e uma camada de apresentação em React, permitindo executar a matriz experimental de forma reproduzível e com registros versionados.

Os resultados preliminares indicam que a troca do servidor RTMP e do transcodificador altera significativamente o tempo de startup do HLS e o perfil de consumo de recursos. O conjunto atual de scripts opera em CPU, sem aceleração por GPU, o que simplifica a replicação do experimento e torna a leitura dos resultados diretamente associada ao host documentado.

Palavras-chave: RTMP; transcodificação; streaming de vídeo; Docker; HLS; desempenho.

1 Introdução

Streaming ao vivo combina requisitos de baixa latência, estabilidade operacional e eficiência no uso de recursos. Em cenários acadêmicos e de produção, pequenas mudanças no servidor de ingestão ou no transcodificador podem alterar o comportamento observável pelo espectador, principalmente quando há múltiplos perfis de saída e vários consumidores simultâneos. Este projeto parte dessa premissa para comparar, em condições controladas, duas opções de servidor RTMP e dois transcodificadores amplamente adotados na prática.

A contribuição inicial do trabalho não está apenas na implementação da plataforma, mas também na organização dos artefatos para escrita científica. Os resultados brutos, os agregados estatísticos, os logs e o relatório consolidado foram estruturados para que cada seção do TCC possa ser derivada do mesmo conjunto de evidências. Isso reduz retrabalho e melhora a rastreabilidade entre experimento, análise e redação.

Do ponto de vista técnico, a versão atual da solução trabalha com transcodificação em software. FFmpeg e GStreamer aparecem como opções alternativas para o mesmo fluxo de entrega HLS, mas sem uso de GPU no pipeline. Esse detalhe é importante porque a interpretação de CPU e memória depende do host, enquanto a comparação entre cenários deve ocorrer dentro do mesmo ambiente documentado.

1.1 Objetivo geral

Avaliar e comparar o comportamento de servidores RTMP e transcodificadores em uma plataforma de streaming ao vivo, observando latência, consumo de recursos, erros de entrega e estabilidade operacional.

1.2 Objetivos específicos

- Documentar a arquitetura de software e infraestrutura usada no experimento.

- Organizar os dados de benchmark em formato apropriado para escrita acadêmica.
- Comparar Nginx-RTMP e SRS com FFmpeg e GStreamer sob cargas distintas.
- Consolidar os resultados recentes em tabelas que possam ser reaproveitadas no texto final.

2 Fundamentação técnica

O fluxo do trabalho é composto por ingestão RTMP, transcodificação em múltiplas variantes e disponibilização via HLS. O padrão HLS é descrito no RFC 8216, que orienta a segmentação do vídeo e a composição de playlists adaptativas [1]. Na prática, a solução utiliza servidores e transcodificadores distintos para isolar o impacto de cada componente no desempenho final.

O FFmpeg é utilizado como transcodificador por sua ampla adoção e flexibilidade em pipelines de áudio e vídeo [2]. Já o GStreamer oferece uma abordagem de construção por elementos, com boa capacidade de composição de fluxos multimídia [3]. No lado da ingestão, Nginx-RTMP fornece uma base enxuta e tradicional para RTMP, enquanto o SRS oferece um servidor especializado em streaming em tempo real [4, 5].

O projeto está empacotado em containers Docker, o que facilita o isolamento do ambiente e a repetição dos testes [6]. O backend em Spring Boot coordena parte do ciclo de vida da aplicação e expõe a camada de controle usada pelos scripts de benchmark [7]. O frontend, por sua vez, atua como superfície de reprodução e observação do comportamento do HLS durante os testes.

2.1 Leitura correta das métricas

As métricas de CPU e memória precisam ser lidas sempre em relação ao host documentado, porque o percentual de CPU do container pode ultrapassar 100% quando há execução concorrente em múltiplas threads. Isso não indica erro por si só; indica a intensidade de uso agregada pelo container. A mesma lógica vale para memória: o consumo observado deve ser comparado com a capacidade total do equipamento e com a carga dos demais serviços em execução.

No conjunto atual, não há uso de GPU no pipeline. Portanto, qualquer ganho ou perda de desempenho observada nos resultados deve ser atribuída ao comportamento do software, ao ambiente Docker e ao hardware do host, e não a aceleração por dispositivos gráficos.

3 Metodologia experimental

A metodologia foi desenhada para que cada cenário pudesse ser executado, medido e reexecutado com o mínimo de ambiguidade. A base da matriz contém duas variáveis principais de controle: o servidor RTMP e o transcodificador. Em seguida, a carga é variada por número de espectadores sintéticos, produzindo um conjunto comparável de combinações.

O benchmark executa a stack por meio de Docker Compose, publica uma stream de teste, gera consumidores sintéticos e coleta as métricas a partir do container RTMP. Os resultados são escritos em CSV bruto, agregados por combinação e consolidados em uma planilha final. O objetivo não é apenas registrar números, mas preservar a trilha de auditoria entre execução, consolidação e interpretação.

3.1 Recalibração dos critérios de aceitação

Os valores de aceite devem refletir o comportamento esperado do sistema implementado. Neste projeto, os scripts de transcodificação operam com `HLS_SEGMENT_DURATION=6` s. Assim, o startup de HLS precisa ser interpretado em função do tempo de geração dos primeiros segmentos e do bootstrap do transcodificador.

Além disso, o benchmark coleta CPU do container RTMP em percentual Docker, onde 100% representa um núcleo lógico. Como o pipeline gera múltiplas variantes em software, valores acima de 100% são esperados e não caracterizam falha por si só.

Com base no código e no conjunto `results-aggregated.csv` mais recente, os limites foram recalibrados com regra de percentis para este ambiente (Ubuntu 24.04.3, i5-13420H, 24 GB RAM):

Tabela 1: Critérios de aceite recalibrados para o estudo

Métrica	Critério proposto
Startup HLS média	Até 24 s (equivale a 4 segmentos de 6 s; cobre o bootstrap observado em Nginx)
CPU média (1 e 10 viewers)	Até 240% (aproxima o P90 observado para baixa/média carga)
CPU média (50 viewers)	Até 280% (faixa de estresse; próximo ao P90 da carga alta)
Taxa de erros (1 e 10 viewers)	Até 1%
Taxa de erros (50 viewers)	Até 3% (faixa de estresse, mantendo controle de degradação)
Reinícios de sessão	Igual a 0

Para análise científica, recomenda-se usar duas leituras simultâneas: (i) conformidade por critérios recalibrados e (ii) comparação relativa entre cenários por média e desvio padrão. Dessa forma, o estudo mantém rigor operacional sem mascarar diferenças de desempenho entre arquiteturas.

3.2 Matriz de comparação

O estudo considera as combinações Nginx-RTMP + FFmpeg, Nginx-RTMP + GStreamer, SRS + FFmpeg e SRS + GStreamer. Cada uma é executada sob três cargas: 1, 10 e 50 espectadores. Isso permite verificar tanto a estabilidade em baixa carga quanto a degradação sob aumento de consumo.

3.3 Fontes de dados

Os arquivos em `app/benchmark/results/latest` são a fonte de verdade para os resultados mais recentes. A partir deles, o artigo pode reaproveitar:

- `results.csv`, com todas as execuções brutas.
- `results-aggregated.csv`, com médias, desvios padrão e verificação dos critérios.
- `resultado.csv`, com a matriz consolidada em formato de apresentação.
- `summary.txt`, com o resumo rápido da execução.

Essa organização reduz o risco de inconsistência entre o que foi medido e o que será descrito no texto final.

4 Ambiente de testes e resultados preliminares

O experimento foi executado em Ubuntu 24.04.3 LTS, kernel 6.17.0-23-generic, com processador Intel Core i5-13420H, 24 GB de RAM e armazenamento NVMe SSD. A orquestração ficou a cargo do Docker Engine, em rede bridge local. Esse contexto é essencial porque as métricas observadas dependem do host e da sobrecarga simultânea dos containers.

Tabela 2: Resumo do ambiente utilizado no benchmark

Item	Especificação
Sistema operacional	Ubuntu 24.04.3 LTS (x86_64), kernel 6.17.0-23-generic
CPU	Intel Core i5-13420H, 13 ^a geração, 12 threads
RAM total	24 GB
Armazenamento	NVMe SSD (231 GB)
Rede	Loopback / bridge Docker
Virtualização	Docker Engine

O último conjunto agregado indica que o servidor influencia diretamente o startup do HLS, enquanto o transcodificador altera a taxa de consumo e a estabilidade. Em termos de leitura prática, os cenários com Nginx exibiram startup acima do limite definido na matriz, enquanto SRS reduziu o tempo para disponibilização do fluxo. Ainda assim, todos os cenários permaneceram pressionados pelo uso de CPU, o que reforça o caráter de comparação do estudo.

4.1 Como ler os resultados de falha

O rótulo FAIL significa que o cenário violou pelo menos um critério de aceitação do plano. Ele não indica erro de execução do benchmark. Na leitura do estudo, cada combinação deve ser classificada pelo critério violado mais relevante para a conclusão.

Tabela 3: Síntese dos resultados agregados mais recentes por combinação

Combinação	Startup HLS	CPU média	RAM média	Bitrate efetivo	Falha principal
Nginx + FFmpeg	21,7–22,3 s	159–197%	1,64–1,68 GB	3522–3526 kbps	Conforme no critério recalibrado
Nginx + GStreamer	22,0 s	233–292%	623–657 MB	5343–5345 kbps	Alerta em 50 viewers por CPU acima de 280%
SRS + FFmpeg	8,0 s	195–216%	1,62–1,68 GB	3155–3158 kbps	Conforme no critério recalibrado
SRS + GStreamer	1,0–7,0 s	192–216%	501–642 MB	5338–5340 kbps	Alerta em 50 viewers por taxa de erros acima de 3%

O arquivo `resultado.csv` da última execução confirma que, com os critérios recalibrados para o contexto deste estudo, a maior parte da matriz fica conforme. Os alertas concentram-se em carga alta (50 viewers), especialmente em CPU para Nginx + GStreamer e em taxa de erros para SRS + GStreamer.

5 Discussão

Os resultados sugerem que o custo de disponibilização inicial do HLS não depende apenas do número de espectadores, mas também da interação entre o servidor RTMP e o transcodificador. Nginx apresentou maior atraso para iniciar a entrega do conteúdo, enquanto SRS respondeu mais rapidamente. Em contrapartida, o uso de CPU permaneceu elevado em todos os cenários, o que mostra que o gargalo do experimento está concentrado na transcodificação em software e na multiplicidade de variantes geradas.

Outro ponto relevante é que a ausência de GPU no pipeline simplifica a interpretação do experimento, mas também delimita o alcance das conclusões. O estudo mede o desempenho realista de uma arquitetura CPU-only, que continua sendo comum em ambientes de laboratório, em provedores de menor porte e em cenários de custo controlado. Caso a evolução do trabalho inclua aceleração por GPU, os parâmetros de comparação precisarão ser reabertos, porque a natureza do gargalo muda de forma significativa.

Do ponto de vista de escrita acadêmica, o material já está organizado para sustentar a seção de método, a seção de resultados e parte da discussão. Isso permite separar com clareza o que foi medido, o que foi agregado estatisticamente e o que é inferência analítica baseada nos números produzidos pelo benchmark.

6 Conclusão

A primeira versão do artigo já pode ser estruturada sobre um conjunto consistente de evidências, com dados de benchmark versionados e documentação técnica reaproveitável. O principal achado preliminar é que o servidor RTMP influencia fortemente o tempo de startup do HLS, enquanto o transcodificador impacta o perfil de consumo e a estabilidade sob carga.

Como encaminhamento imediato, o texto final pode expandir três frentes: fundamentação teórica, análise comparativa dos resultados e discussão sobre limitações do ambiente CPU-only. Também vale registrar, de forma explícita, que a leitura das métricas depende do host documentado e que o conjunto atual não emprega GPU.

Com isso, a base em LaTeX fica pronta para evoluir de um template inicial para o corpo completo do TCC, sem necessidade de reestruturar novamente os resultados já produzidos.

Referências

- [1] Pantos, R.; May, W. *HTTP Live Streaming*. RFC 8216, IETF, 2017.
- [2] FFmpeg Project. *FFmpeg Documentation*. Disponível em: <https://ffmpeg.org/documentation.html>. Acesso em: 18 maio 2026.
- [3] GStreamer Project. *GStreamer Documentation*. Disponível em: <https://gstreamer.freedesktop.org/documentation/>. Acesso em: 18 maio 2026.
- [4] Arut. *nginx-rtmp-module*. Repositório GitHub. Disponível em: <https://github.com/arut/nginx-rtmp-module>. Acesso em: 18 maio 2026.
- [5] SRS Project. *Simple Realtime Server Documentation*. Disponível em: <https://ossrs.io/>. Acesso em: 18 maio 2026.
- [6] Docker Inc. *Docker Documentation*. Disponível em: <https://docs.docker.com/>. Acesso em: 18 maio 2026.
- [7] VMware Tanzu. *Spring Boot Reference Documentation*. Disponível em: <https://docs.spring.io/spring-boot/>. Acesso em: 18 maio 2026.
- [8] Projeto TCC. *Benchmark RTMP e resultados consolidados*. Arquivos em `app/benchmark/results/latest`, 2026.